Rapport de TP : Système de Traitement de Commandes Distribué avec RabbitMQ 1
1. Installation, Environnement et Configuration du Broker 1
2. Déclaration de la Topologie de Messagerie (Topologie.java) 1
Justification des paramètres de configuration : 2
3. Producteur, Confirmation de Publication et Partage de Charge (Fair Dispatch) 2
Publication sécurisée (Producteur.java) 3
Distribution de charge (Fair Dispatch) avec basicQos(1) 3
4. Garantie de Livraison, Accusés Manuels et Simulation de Panne 5
Expérience d'interruption du Worker (Ctrl+C) 5
Constat et Analyse 5
5. Routage Avancé (Topic et Fanout) 6
A. Filtrage par Topic (tp.journaux) 6
B. Diffusion Fanout (tp.evenements) 7
6. Gestion des Échecs et Dead Letter Queues (DLQ) 9
Configuration du Dead Letter Exchange (DLX) et de la DLQ 9
Stratégie de rejet sans requeue (basicNack) 10
Observation des résultats 10

Rapport de TP : Système de Traitement de Commandes Distribué avec RabbitMQ

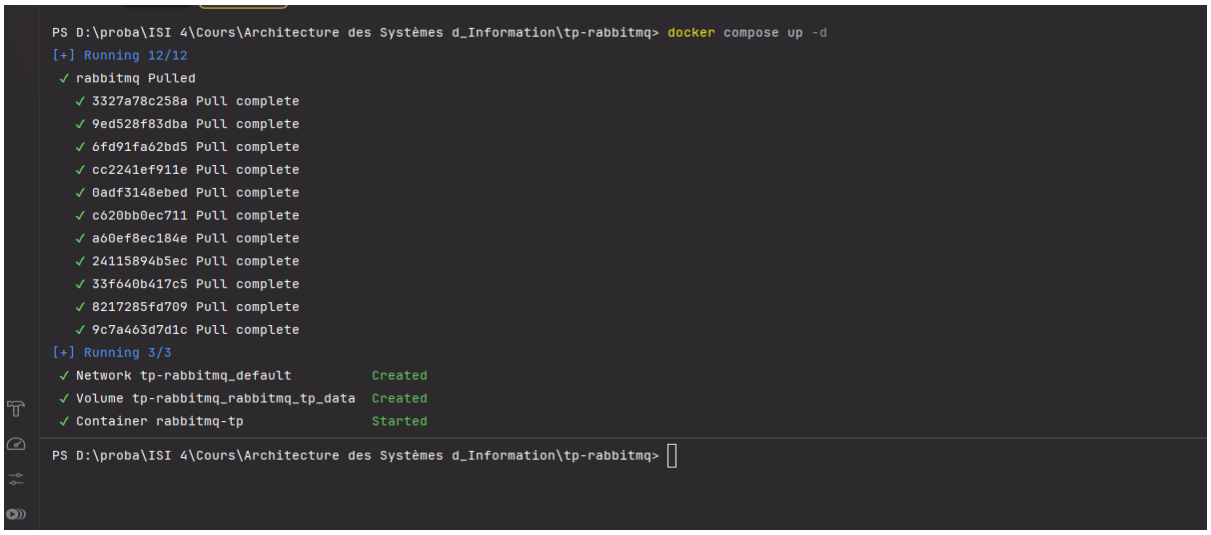
1. Installation, Environnement et Configuration du Broker

L'infrastructure de messagerie repose sur le déploiement d'un broker **RabbitMQ 4.3.6** avec son module de gestion (`management`) conteneurisé sous Docker. La communication s'effectue via le protocole AMQP 0.9.1 sur le port standard 5672, tandis que la console de supervision Web est accessible sur le port 15672.

Le port 15672 est strictement dédié au protocole HTTP/REST pour l'interface de gestion et les métriques de diagnostic. Il ne peut pas recevoir de connexions AMQP binaires (qui nécessitent le port 5672), car les deux protocoles diffèrent à la fois sur la couche de transport et sur le format des trames de communication.

L'accès et la persistance des données du broker sont assurés par un volume Docker nommé `rabbitmq_tp_data`, garantissant que les configurations, échanges et messages conservent leur état même après l'arrêt ou le redémarrage des conteneurs.

- Déploiement et lancement du conteneur Docker RabbitMQ avec initialisation du volume nommé.



2. Déclaration de la Topologie de Messagerie (`Topologie.java`)

La structure du système de messagerie a été initialisée de façon programmatique à l'aide d'une classe de topologie centrale. Trois types d'exchanges ont été déclarés afin de couvrir l'ensemble des besoins de routage :

1. `tp.commandes` **(**`direct`**) :** Transmet les messages d'exécution métier directement à la file abonnée en fonction d'une clé exacte.

2. `tp.journaux` **(**`topic`**) :** Filtre et achemine les logs en fonction de motifs (*patterns*) hiérarchiques séparés par des points.

3. `tp.evenements` **(**`fanout`**) :** Diffuse chaque événement métier à l'ensemble des files abonnées en ignorant la clé de routage.

Les files d'attente ont été configurées en mode classique (`x-queue-type: classic`) et durables (`durable = true`) :

- `tp.preparation` : liée à `tp.commandes` via la clé `preparation`.

- `tp.notifications` et `tp.statistiques` : liées à `tp.evenements` avec une clé vide `""`.

- `tp.logs.commandes` : liée à `tp.journaux` avec le motif `commande.*`.

- `tp.logs.erreurs` : liée à `tp.journaux` avec le motif `#.erreur`.

Justification des paramètres de configuration :

- `durable = true` : Garantit la métadonnée et la structure de la file sur le disque en cas de crash du broker.

- `exclusive = false` : Autorise la connexion simultanée de plusieurs consommateurs à la même file.

- `autoDelete = false` : Empêche la suppression automatique de la file lorsque le dernier worker se déconnecte.

3. Producteur, Confirmation de Publication et Partage de Charge (*Fair Dispatch*)

Publication sécurisée (`Producteur.java`)

Le producteur s'assure qu'aucun message n'est perdu lors de son envoi :

- **Persistance (`deliveryMode = 2`)** : Écrit le corps du message sur le disque.
- **Confirmations (`ch.confirmSelect()`)** : Bloque jusqu'à la réception de l'accusé de réception envoyé par le broker via `waitForConfirmsOrDie()`.
- **Routage obligatoire (`mandatory = true`)** : Déclenche un `ReturnListener` si aucune file n'est liée à la clé de publication.

Distinction `mandatory` vs `confirmSelect` :

- La confirmation (`confirmSelect`) atteste uniquement que le broker a bien reçu le message et l'a pris en charge.
- L'option `mandatory` garantit que le message a été effectivement déposé dans au moins une file. Un message peut être confirmé par le broker tout en déclenchant un retour `NO_ROUTE` s'il n'existe aucune file destinataire.

Distribution de charge (*Fair Dispatch*) avec `basicQos(1)`

Pour éviter qu'un consommateur plus lent n'accumule un lot de messages au détriment des autres, le mécanisme de contrôle de flux impose un `prefetchCount` égal à **1**.

Dans nos essais avec 20 commandes (`m-001` à `m-020`) réparties sur deux workers (`W1` et `W2`), la charge s'est équilibrée dynamiquement : dès qu'un worker acquitte un message (`basicAck`), RabbitMQ lui délivre le message suivant. Si la vitesse de traitement varie d'un worker à l'autre (ex: 1000 ms contre 4000 ms), le worker le plus rapide absorbe une part proportionnellement plus importante du lot.

- Confirmation de publication de la série de messages par le producteur.

```
PS C:\Users\mirre\Pictures\tp-rabbitmq> mvn exec:java "-Dexec.mainClass=tp.Producteur" "-Dexec.args=tp.commandes preparation"
[INFO] Scanning for projects...
[INFO]
[INFO] ---------------------------< tp:tp-rabbitmq >---------------------------
[INFO] Building tp-rabbitmq 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec:3.6.4:java (default-cli) @ tp-rabbitmq ---
Publication confirmee: m-001
Publication confirmee: m-002
Publication confirmee: m-003
Publication confirmee: m-004
Publication confirmee: m-005
Publication confirmee: m-006
Publication confirmee: m-007
Publication confirmee: m-008
Publication confirmee: m-009
Publication confirmee: m-010
Publication confirmee: m-011
Publication confirmee: m-012
Publication confirmee: m-013
Publication confirmee: m-014
```

- Execution du worker `W2` traitant ses commandes associées.

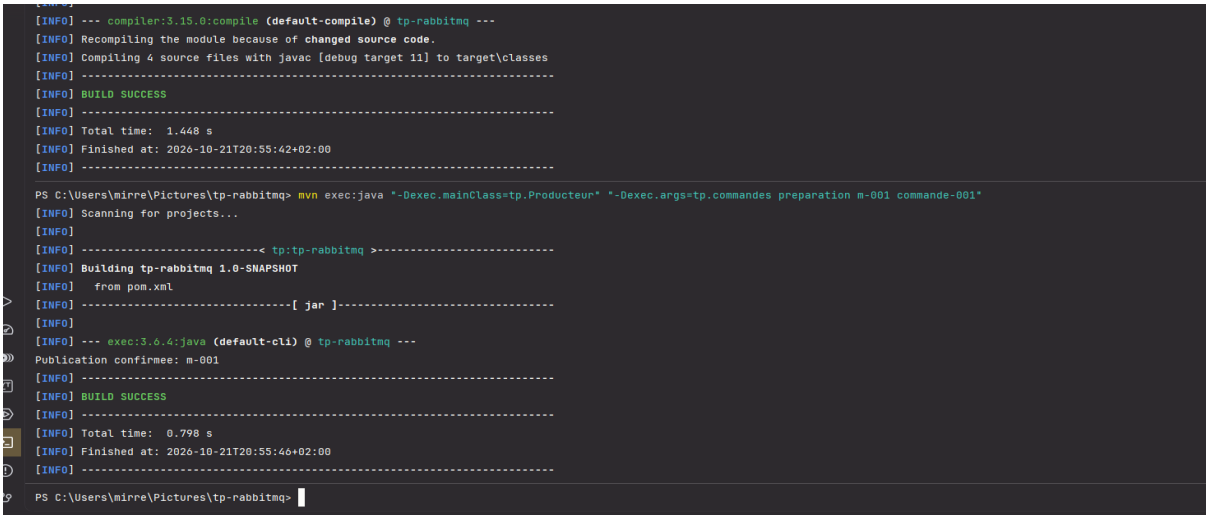
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\mirre\Pictures\tp-rabbitmq> mvn exec:java "-Dexec.mainClass=tp.Worker" "-Dexec.args=tp.preparation W2 2000"
[INFO] Scanning for projects...
[INFO]
[INFO] ---------------------------< tp:tp-rabbitmq >---------------------------
[INFO] Building tp-rabbitmq 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec:3.6.4:java (default-cli) @ tp-rabbitmq ---
W2 attend sur tp.preparation
W2 DEBUT id=m-002 redelivered=false corps=commande-002
W2 FIN id=m-002
W2 DEBUT id=m-003 redelivered=false corps=commande-003
W2 FIN id=m-003
W2 DEBUT id=m-005 redelivered=false corps=commande-005
W2 FIN id=m-005
W2 DEBUT id=m-007 redelivered=false corps=commande-007
W2 FIN id=m-007
W2 DEBUT id=m-009 redelivered=false corps=commande-009
```

- Execution du worker `W1` traitant ses commandes associées.

```
PS C:\Users\mirre\Pictures\tp-rabbitmq> mvn exec:java "-Dexec.mainClass=tp.Worker" "-Dexec.args=tp.preparation W1 2000"
[INFO] Scanning for projects...
[INFO]
[INFO] ---------------------------< tp:tp-rabbitmq >---------------------------
[INFO] Building tp-rabbitmq 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec:3.6.4:java (default-cli) @ tp-rabbitmq ---
W1 attend sur tp.preparation
W1 DEBUT id=m-001 redelivered=false corps=commande-001
W1 FIN id=m-001
```

- Journal d'exécution du premier message traité par le worker `W1`.



4. Garantie de Livraison, Accusés Manuels et Simulation de Panne

La fiabilité de l'application repose sur la désactivation de l'acquittement automatique (`autoAck = false`). Le consommateur traite le message puis appelle explicitement `ch.basicAck()`.

Expérience d'interruption du Worker (`Ctrl+C`)

Pour vérifier la résistance du système face aux pannes d'infrastructure :

1. Un message `m-001` a été envoyé à la file `tp.preparation`.

2. Un worker configuré avec un délai de traitement long (`W-Panne`) a réceptionné le message (`Unacked = 1`).

3. Le processus du worker a été brusquement interrompu via un signal `Ctrl+C` avant l'exécution du `basicAck`.

Constat et Analyse

Dès la fermeture brutale du canal AMQP, RabbitMQ a détecté la perte de connexion. Le message est immédiatement repassé de l'état `Unacked` à l'état `Ready`. Lors de sa réattribution ultérieure à un nouveau worker, le drapeau `redelivered` est passé à `true`. Cet indicateur permet à l'application de savoir que le message a déjà fait l'objet d'une tentative d'exécution incomplète.

- Lancement du worker `W-Panne` en attente de traitement sur la file `tp.preparation`.



5. Routage Avancé (Topic et Fanout)

A. Filtrage par Topic (`tp.journaux`)

Le filtrage d'événements s'appuie sur la syntaxe à points des clés de routage :

- remplace exactement **un mot**.
- remplace **zéro ou plusieurs mots**.

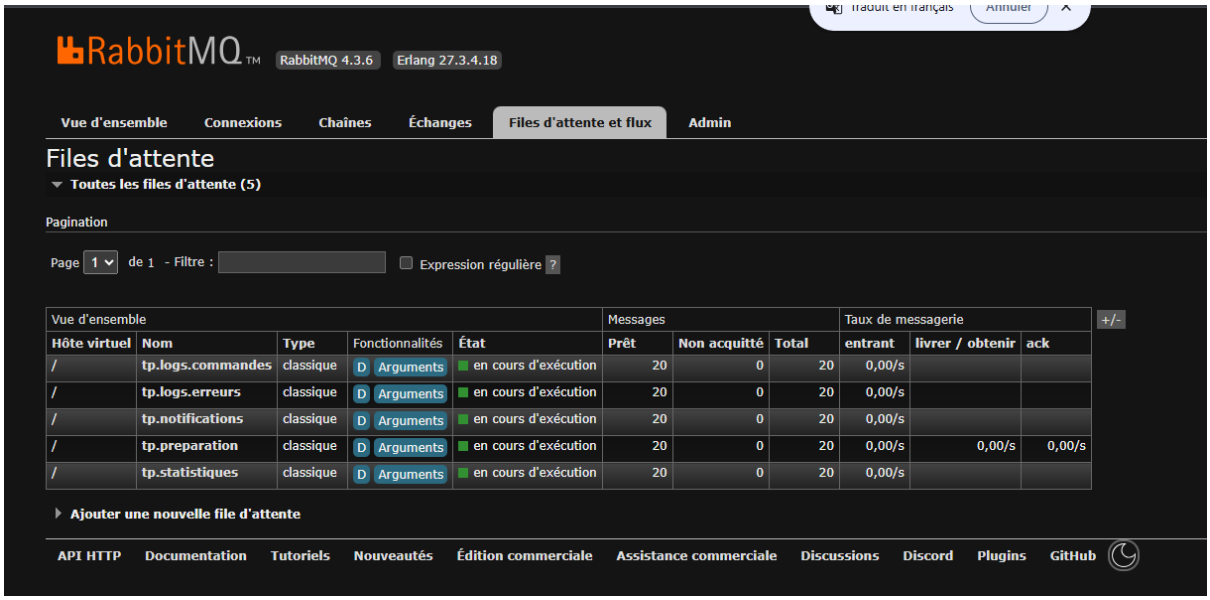
Clé de routage	tp.logs.commandes (commande.*)	tp.logs.erreurs (#.erreur)	Interprétation
`commande.cree`	**Reçu**	Non	Match `commande.*` (un mot après le point).
`paiement.erreur`	Non	**Reçu**	Match `#.erreur` (se termine par `.erreur`).
`commande.erreur`	**Reçu**	**Reçu**	Match les deux règles simultanément. Dupliqué dans les deux files.

`commande.stock.reserve`	Non	Non	Deux mots après le point : rejeté par `commande.*` (aurait demandé `commande.#`).

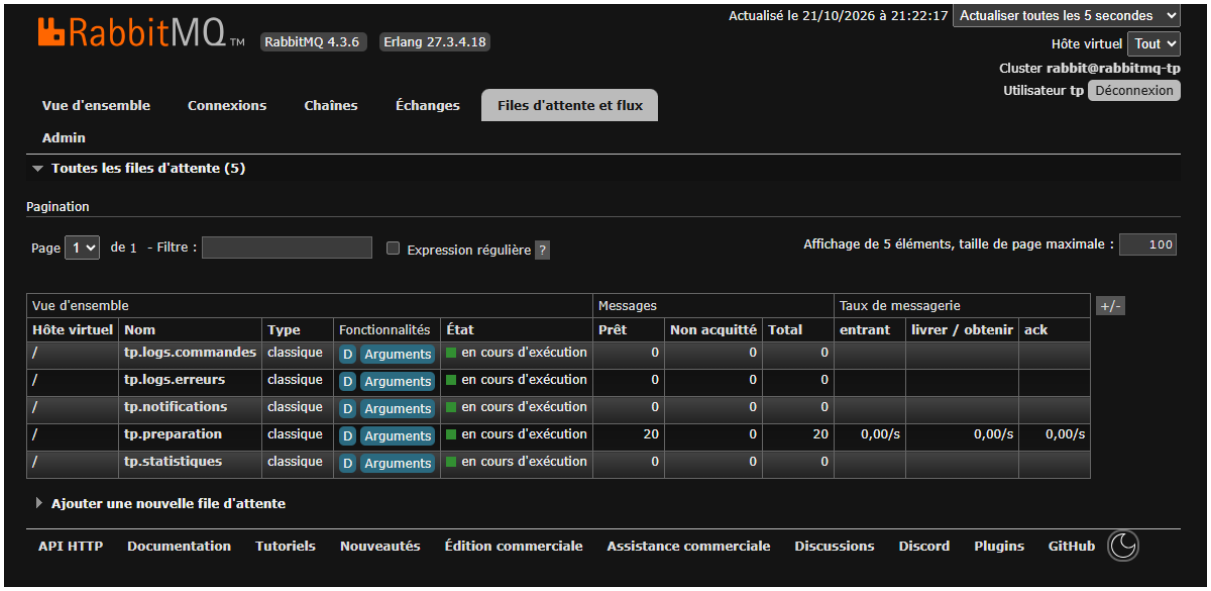
B. Diffusion Fanout (`tp.evenements`)

Lors de la publication de 20 événements sur l'échange `tp.evenements` avec la clé de routage `broadcast` :

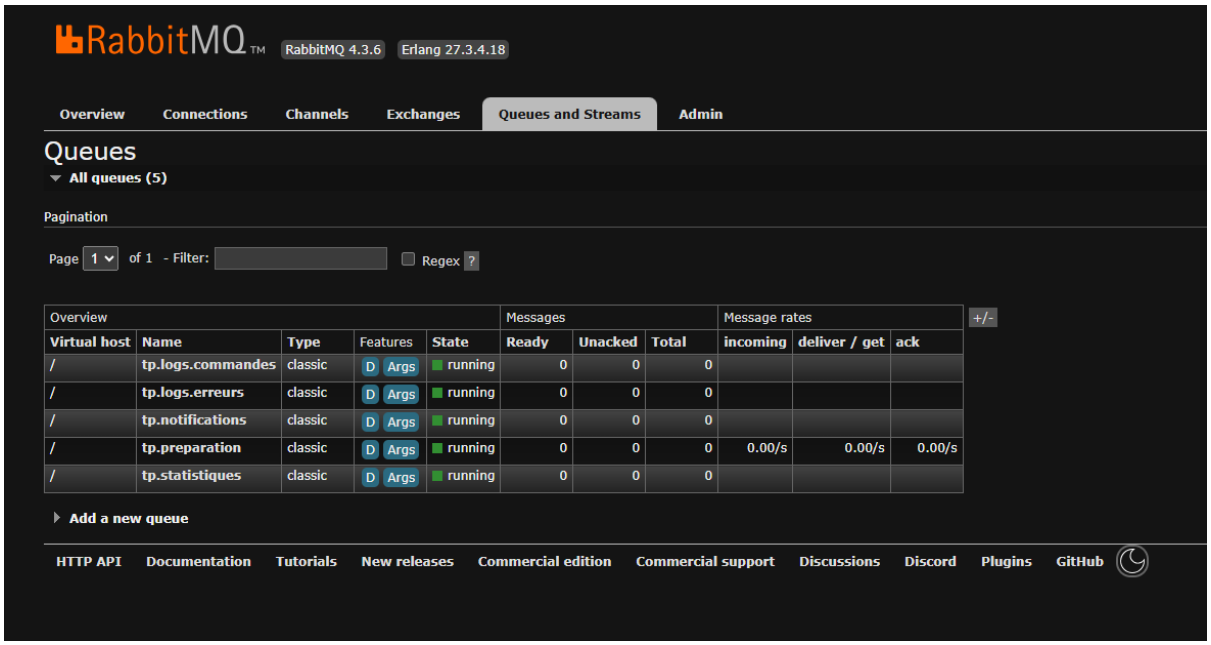
- L'échange *fanout* a ignoré la clé de routage.
- Les 20 messages ont été dupliqués et acheminés simultanément vers les files `tp.notifications` et `tp.statistiques`.
- État des files de logs et de notifications dans la console RabbitMQ affichant les messages prêts dans chaque file.



- `02.` Vue d'ensemble montrant les compteurs de messages prêts (`Ready = 20`) dans la file `tp.preparation`.



- Console d'administration RabbitMQ montrant l'état vide des files d'attente une fois l'ensemble des traitements acquittés par les workers.



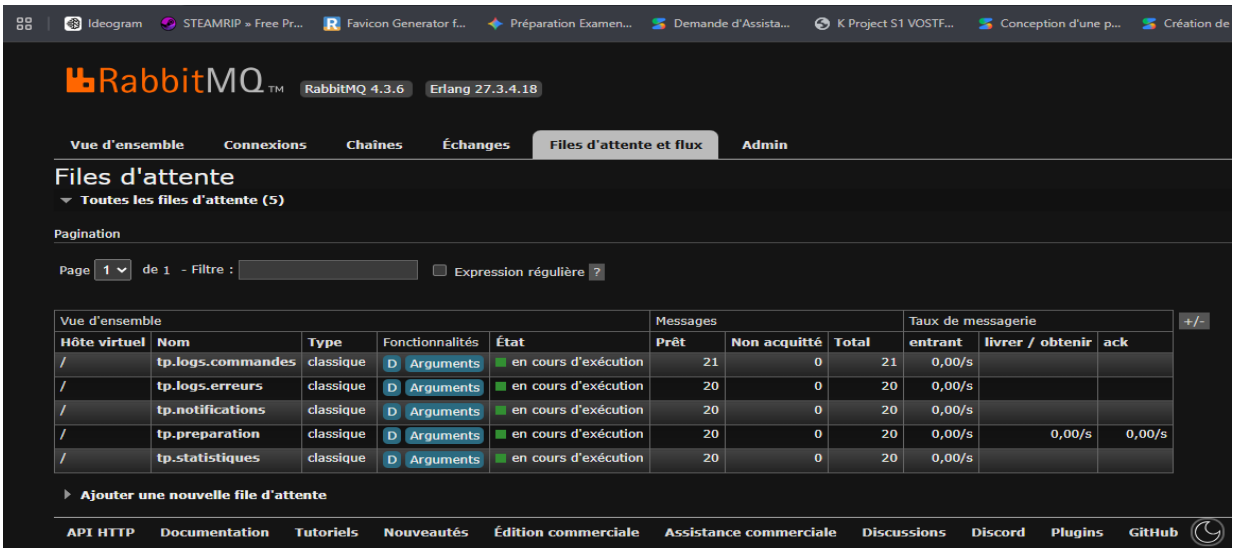
Lors de la publication d'un message avec la clé de routage `commande.creee` :

- L'échange de type *topic* (`tp.journaux`) évalue le motif configuré sur les différentes files de logs.
- La file **`tp.logs.commandes`**, liée via le motif `commande.*`, intercepte et stocke le message correspondant.

La console d'administration atteste de ce comportement : le compteur de la file de logs des commandes passe à 21 messages prêts, validant l'exactitude du filtrage thématique par rapport aux autres files du système.

Capture d'écran à insérer pour cette section :

- Console RabbitMQ montrant l'état des files d'attente avec la file **`tp.logs.commandes`** à 21 messages prêts (`Prêt = 21`).



Voici la rédaction officielle de la section concernant la gestion des échecs et la mise en place des Dead Letter Queues (DLQ), prête à être intégrée dans votre rapport avec la référence à la capture que vous venez de réaliser :

6. Gestion des Échecs et Dead Letter Queues (DLQ)

Dans un système de messagerie d'entreprise, la gestion des messages invalides ou non traitables (*poison messages*) est essentielle pour éviter qu'un consommateur ne se bloque en boucle infinie sur un traitement en erreur.

Configuration du Dead Letter Exchange (DLX) et de la DLQ

Pour isoler les anomalies sans perturber le fonctionnement des files de production principales, une architecture de rebond a été mise en place :

1. **Exchange de secours (`tp.echecs`) :** Déclaré en mode direct pour recevoir les messages rejetés.

2. **File de mise au rebut (`tp.preparation.dlq`) :** Liée à l'exchange `tp.echecs` via la clé de routage `preparation.echec`.

3. **File robuste (`tp.preparation.robuste`) :** Paramétrée avec les arguments de configuration AMQP pointant vers le DLX :
   Map.of(
   "x-queue-type", "classic",

```
        "x-dead-letter-exchange", "tp.echecs",
        "x-dead-letter-routing-key", "preparation.echec"
    )
```

Stratégie de rejet sans requeue (`basicNack`)

Lorsqu'un worker (`WorkerRobuste`) analyse un message et détecte un format corrompu ou invalide, il émet un accusé de rejet explicite en positionnant le paramètre `requeue` à `false` :

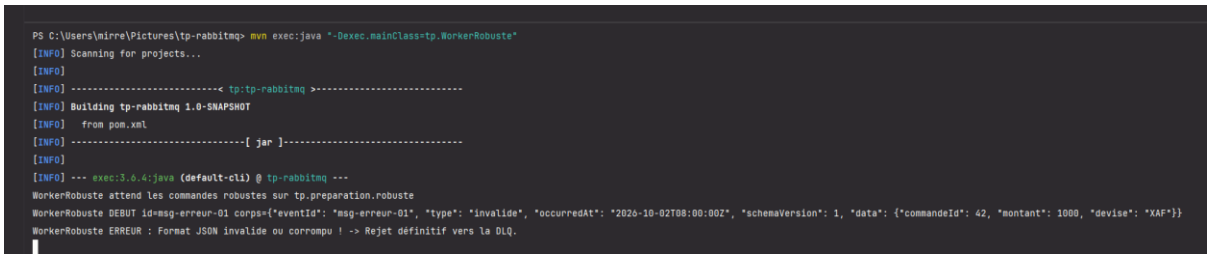
```
ch.basicNack(deliveryTag, false, false);
```

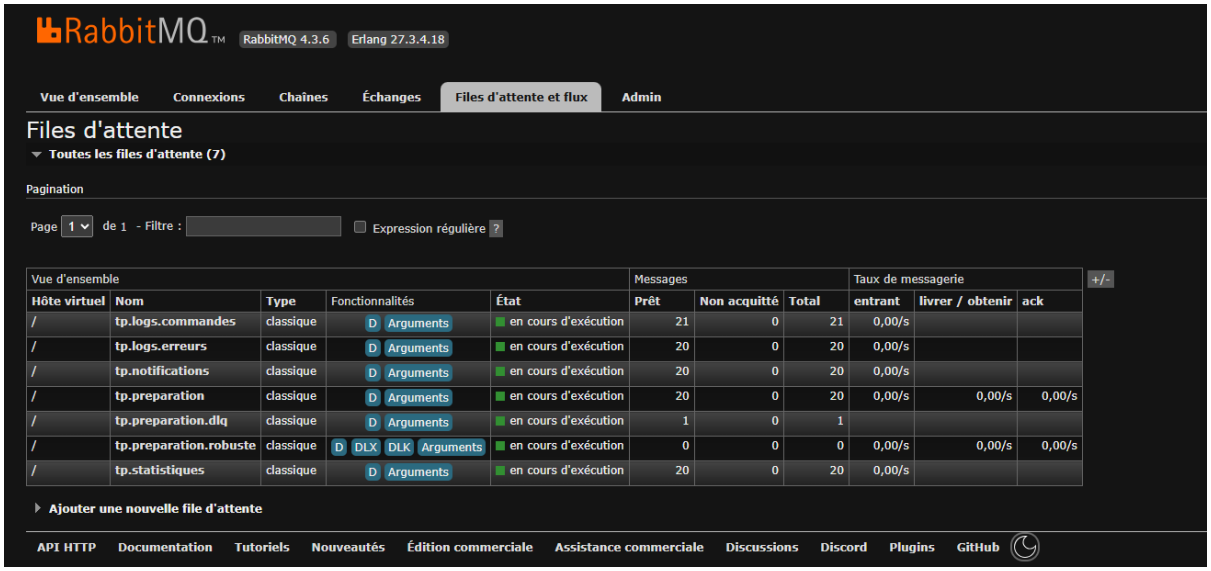
Ce refus définitif indique au broker de ne pas réinsérer le message dans la file d'origine, provoquant son transfert automatique et immédiat vers la file de rebond (`tp.preparation.dlq`).

Observation des résultats

Lors des tests pratiques avec l'envoi d'un message comportant un corps invalide, le consommateur robuste a rejeté le message. L'interface d'administration RabbitMQ atteste que le message a quitté la file robuste pour s'accumuler dans la file dédiée aux échecs, garantissant ainsi la traçabilité des anomalies.

- Console d'administration RabbitMQ montrant la file `tp.preparation.dlq` avec 1 message prêt (`Prêt = 1`), tandis que la file `tp.preparation.robuste` est vide.





Dépot github:
https://github.com/jnguetsa/tp-RabbitMQ.git